

De Kaggle à une simulation distribuée : Étude comparative entre MongoDB (Change Streams) et Redis (Pub/Sub)

Marewane Fertikh

BUT3 Informatique — IUT de Villetaneuse

Ressource : Nouveaux Paradigmes de Bases de Données

Encadrant : M. Cerin

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Objectifs	5
1.3	Approche générale	5
1.4	Résultats majeurs	5
2	Matériel et méthodes	6
2.1	Environnement de développement	6
2.2	Organisation du projet	6
2.3	Activation du Replica Set MongoDB	6
2.4	Installation des dépendances Python	7
3	Préparation des données	8
3.1	Origine du dataset	8
3.2	Nettoyage et adaptation	8
3.3	Conversion vers JSON	8
3.4	Structure finale des données	8
3.5	Sélection aléatoire pour la simulation	9
4	Implémentation MongoDB	10
4.1	Modèle documentaire	10
4.2	Change Streams	10
4.3	Scénario d'exécution MongoDB	11
4.4	Délais réalistes et lisibilité	12
4.5	Logs colorés pour le débogage	12
4.6	Problèmes rencontrés	12
4.7	Diagramme de séquence du pipeline global	12
4.8	Extraits de code essentiels	13
4.8.1	MongoDB : écoute d'un Change Stream	13
4.8.2	Redis : publication et réception d'un événement	14

5	Implémentation Redis	15
5.1	Objectif de l'architecture Redis	15
5.2	Architecture Pub/Sub	15
5.3	Stockage minimal en mémoire	15
5.4	Acteur 1 : le client	16
5.5	Acteur 2 : le restaurant	16
5.6	Acteur 3 : la plateforme	16
5.7	Acteur 4 : les livreurs	16
5.8	Délais réalistes et synchronisation	17
5.9	Logs colorés transversaux	17
5.10	Problèmes rencontrés	17
6	Résultats	18
6.1	Résultats MongoDB	18
6.1.1	Captation des événements : fiabilité des Change Streams	18
6.1.2	Délais simulés et comportement observable	18
6.1.3	Persistance durable et historisation	18
6.2	Résultats Redis	18
6.2.1	Latence très faible et exécution instantanée	18
6.2.2	Aucune perte de message dans le cadre du PoC	19
6.2.3	Gestion correcte de la concurrence entre livreurs	19
6.2.4	Logs colorés multi-acteurs	19
6.3	Comparaison globale des deux approches	19
6.4	Synthèse des résultats	19
7	Analyse comparative	20
7.1	Latence et réactivité	20
7.2	Persistance et cohérence des données	20
7.3	Historisation et traçabilité	20
7.4	Robustesse multi-acteurs et gestion de la concurrence	21
7.5	Facilité de mise en œuvre	21
7.6	Synthèse comparative	21
7.7	Conclusion analytique	21
8	Discussion	23
8.1	Validité du protocole expérimental	23
8.2	Réalisme du modèle Uber Eats simulé	23
8.3	Limites techniques observées	23
8.4	Enseignements pour une architecture distribuée moderne	24
8.5	Comparaison avec des solutions industrielles	24
9	Conclusion	25
10	Guide de reproduction (README condensé)	26
10.1	1. Installation du projet	26
10.2	2. Préparation des données	26
10.3	3. Activer le Replica Set MongoDB	26
10.4	4. Lancer le PoC MongoDB	26
10.5	5. Lancer le PoC Redis	26

10.6	6. Vérifier les logs Redis	27
10.7	7. Nettoyer les commandes Redis	27

Résumé

Ce projet compare deux technologies NoSQL dans un scénario inspiré d’Uber Eats : **MongoDB** utilisant les *Change Streams* et **Redis** reposant sur le modèle *Publish/Subscribe*. L’objectif est de simuler un pipeline d’événements où un client crée une commande, un restaurant la prépare et un livreur la livre, avec un suivi temps réel.

Deux enseignements majeurs ressortent :

- **Redis** excelle dans la diffusion instantanée d’événements grâce à son architecture en mémoire ;
- **MongoDB** assure un suivi cohérent, historisé et requêtable du cycle de vie des commandes.

Deux Proofs of Concept indépendants ont été réalisés en Python, chacun exploitant une approche différente du temps réel. Leur comportement, leurs limites et leurs cas d’usage idéaux sont analysés.

1 Introduction

1.1 Contexte

Les plateformes de livraison modernes, comme Uber Eats, reposent sur une chaîne d'événements continue : création d'une commande, affectation à un restaurant, préparation, prise en charge par un livreur, puis livraison. Ce type d'architecture nécessite à la fois :

- une **réactivité forte** pour informer les utilisateurs instantanément ;
- une **persistance fiable** pour conserver un historique cohérent des actions.

Deux technologies NoSQL permettent de couvrir ces enjeux :

- **Redis**, orienté mémoire, spécialisé dans les événements temps réel ;
- **MongoDB**, orienté documents, robuste en persistance et observation de données.

1.2 Objectifs

L'objectif principal est de comparer concrètement les deux approches au travers d'un scénario réaliste simulant une plateforme de livraison. Plus précisément, nous cherchons à :

- implémenter deux Proofs of Concept (PoC) reposant sur Redis et MongoDB ;
- mesurer leur réactivité et leur comportement dans un cycle complet de commande ;
- analyser leurs points forts, faiblesses et usages recommandés ;
- produire une architecture reproductible utilisable en TP ou en démonstration.

1.3 Approche générale

Pour assurer une comparaison équitable, les deux PoC suivent la même logique d'exécution :

1. un client crée une commande ;
2. un restaurant la prépare ;
3. un livreur la prend en charge et la livre.

Les deux implémentations exploitent les mêmes données Kaggle (restaurants et menus), une génération aléatoire des acteurs (restaurants, menus, clients, livreurs) et un pipeline basé sur des événements publiés en temps réel. Chaque PoC est volontairement indépendant afin de faciliter l'analyse et la reproductibilité.

1.4 Résultats majeurs

Deux résultats essentiels structurent la comparaison présentée dans ce rapport :

- **Redis** assure une diffusion d'événements quasi instantanée, ce qui en fait une solution très adaptée aux interactions temps réel entre les acteurs de la plateforme.
- **MongoDB** permet une historisation complète et durable du cycle de vie d'une commande grâce aux *Change Streams*, offrant une traçabilité fine que Redis ne permet pas nativement.

Ces deux observations serviront de fil conducteur à l'analyse et orienteront l'évaluation des deux PoC tout au long du rapport.

2 Matériel et méthodes

2.1 Environnement de développement

L'ensemble du projet a été réalisé dans un environnement contrôlé afin de garantir la reproductibilité :

- **OS** : Ubuntu 24.04 LTS ;
- **Langage** : Python 3.12 avec environnement virtuel ;
- **Bases NoSQL** :
 - MongoDB Community 7.0 (Replica Set activé) ;
 - Redis 7.2 pour le Pub/Sub ;
- **Outils complémentaires** :
 - mongosh pour la gestion du Replica Set ;
 - redis-cli pour les tests Pub/Sub ;
 - Visual Studio Code et NeoVim pour l'édition de code.

Un environnement virtuel Python a été utilisé :

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
```

2.2 Organisation du projet

Le projet est structuré en deux répertoires indépendants, chacun contenant un Proof of Concept complet :

```
1 uubereats-poc/
2     data -> data_local/
3     data_local/
4         restaurants.json
5         menus.json
6         ...
7     mongo/
8         acteurs/
9         models.py
10        db.py
11        ...
12    redis_poc/
13        acteurs/
14        data_loader.py
15        db.py
16        logger.py
17    tools/
```

Listing 1 – Structure du projet

Le dossier `data` pointe vers `data_local` via un lien symbolique afin d'éviter d'inclure dans Git des fichiers lourds (plus de 15 Mo pour les menus Kaggle). Cette approche permet de respecter les limites de GitHub tout en conservant une structure projet propre.

```
1 ln -s data_local data
```

2.3 Activation du Replica Set MongoDB

Les Change Streams de MongoDB nécessitent obligatoirement un Replica Set, même en environnement local. L'activation se fait en trois étapes :

```
1 sudo systemctl stop mongod
2 sudo systemctl start mongod --replSet rs0
3 mongosh
4 > rs.initiate()
```

Une commande de vérification :

```
1 mongosh --eval "rs.status()"
```

Un Replica Set valide est indispensable pour le fonctionnement du PoC MongoDB.

2.4 Installation des dépendances Python

Les deux PoC utilisent les mêmes dépendances principales :

- **pymongo** : connexion MongoDB + Change Streams ;
- **redis-py** : client Redis moderne (Pub/Sub) ;
- **coloredlogs** / **colorama** (optionnel) : pour l’affichage coloré.

L’installation globale se fait via :

```
1 pip install -r requirements.txt
```

Chaque PoC est ensuite lancé via :

```
1 python -m mongo.acteurs.client
2 python -m redis_poc.acteurs.client
```

3 Préparation des données

3.1 Origine du dataset

Les données utilisées proviennent d'un dataset public Kaggle contenant :

- **restaurants.csv** : informations sur les restaurants (nom, adresse, géolocalisation, catégories) ;
- **restaurant-menus.csv** : menus associés à chaque restaurant ;

Ce dataset est suffisamment riche pour simuler un environnement de type Uber Eats, tout en restant exploitable dans un PoC local.

3.2 Nettoyage et adaptation

Les fichiers sources présentaient plusieurs irrégularités :

- certains champs vides ou des chaînes mal formatées ;
- plusieurs restaurants sans menu associé ;
- identifiants non homogènes entre les deux fichiers ;
- structure tabulaire (CSV) peu adaptée au modèle documentaire de MongoDB.

Un script de conversion dédié a donc été créé (`tools/csv_to_json.py`) afin de produire des JSON propres et directement exploitables.

3.3 Conversion vers JSON

```
1 import csv, json
2
3 with open("restaurants.csv") as f:
4     rows = list(csv.DictReader(f))
5
6 with open("restaurants.json", "w") as out:
7     json.dump(rows, out, indent=2)
```

Listing 2 – Extrait du script de conversion CSV → JSON

Les fichiers finaux (`restaurants.json` et `menus.json`) sont stockés dans `data_local/`.

Pour éviter d'inclure plus de 15 Mo de données dans GitHub, un lien symbolique est utilisé :

```
1 ln -s data_local data
```

MongoDB et Redis lisent alors les données via le chemin `data/`, ce qui garantit une compatibilité totale.

3.4 Structure finale des données

Chaque restaurant a la forme :

```
1 {
2     "id": "450",
3     "name": "American Deli",
4     "full_address": "3802 Flat Shoals Rd",
5     "lat": "33.56",
6     "lng": "-84.52",
7     "category": "American, Wings",
8     ...
9 }
```


Les menus sont représentés ainsi :

```
1 {  
2   "restaurant_id": "450",  
3   "menu_id": "691cdb087d9dc0f7dc5b6ffd",  
4   "name": "Burger Meal (Fries and Drink)",  
5   "price": "10.99 USD",  
6   "category": "Meal"  
7 }
```

3.5 Sélection aléatoire pour la simulation

Lors de chaque exécution, un client choisit :

- un restaurant aléatoire ;
- un menu appartenant à ce restaurant ;
- un identifiant client généré pseudo-aléatoirement.

Ce comportement reproduit un trafic réaliste et permet de tester entièrement le pipeline MongoDB et Redis.

Le module `redis_poc/data_loader.py` garantit :

- la cohérence restaurant-menu ;
- l'absence d'erreurs même si un restaurant possède beaucoup ou peu de menus ;
- la reproductibilité du comportement dans les deux PoC.

4 Implémentation MongoDB

L'implémentation MongoDB repose sur trois piliers techniques :

1. un **Replica Set local**, indispensable pour activer les Change Streams ;
2. un **modèle documentaire simple et cohérent** pour les commandes ;
3. quatre acteurs simulés en Python (client, plateforme, restaurant, livreur), échangeant uniquement via la base.

Chaque acteur agit dans son propre terminal, ce qui permet une visualisation fidèle du flux d'évènements d'une application réelle de type Uber Eats.

4.1 Modèle documentaire

MongoDB stocke chaque commande dans une collection unique `commandes`. Le document est volontairement dénormalisé afin de faciliter le suivi du cycle de vie :

```
1 {
2   "_id": ObjectId("..."),
3   "commande_id": "6925ffb10ea6ad2f658ff867",
4   "client_id": "CLIENT_2307",
5   "restaurant_id": "450",
6   "restaurant_name": "American Deli",
7   "menu_id": "691cdb087d9dc0f7dc5b6ffd",
8   "menu_name": "Burger Meal (Fries and Drink)",
9   "livreur_id": null,
10  "statut": "cree",
11  "timestamp": 1764097969.657
12 }
```

Listing 3 – Structure d'une commande MongoDB

Les statuts possibles sont :

cree → preparation → prete → prise_en_charge → livree →
notification_client

Ce schéma linéaire reflète le diagramme de séquence du projet et simplifie l'utilisation des Change Streams.

4.2 Change Streams

MongoDB notifie en temps réel tout changement dans les documents suivis. Dans ce projet, seul le **client** utilise un Change Stream, afin d'être notifié de la livraison finale.

Extrait du code :

```
1 pipeline = [
2   {"$match": {
3     "operationType": "update",
4     "fullDocument.commande_id": commande_id
5   }}
6 ]
7
8 with commandes.watch(pipeline) as stream:
9     for change in stream:
10         new_status = change["fullDocument"]["statut"]
```

```

11         print("    Nouveau statut :", new_status)
12
13     if new_status == "notification_client":
14         print("Commande livr e !")
15         break

```

Listing 4 – Écoute d'un Change Stream dans le client MongoDB

Le client s'arrête automatiquement lorsque le livreur publie le statut final.

4.3 Scénario d'exécution MongoDB

L'exécution se déroule en quatre terminaux distincts, dans le dossier `mongo/acteurs/`.

```
1 python -m mongo.acteurs.client
```

Le client :

- sélectionne un restaurant et un menu aléatoires ;
- crée une commande ;
- écoute les Change Streams ;
- termine lorsque la livraison est confirmée.

```
1 python -m mongo.acteurs.restaurant
```

Le restaurant :

- écoute les commandes créées ;
- simule la préparation (temps aléatoire réaliste) ;
- met à jour le statut à **prete**.

```
1 python -m mongo.acteurs.plateforme
```

La plateforme :

- détecte les commandes prêtes ;
- sélectionne un livreur disponible ;
- affecte la commande ;
- met à jour le statut.

```
1 python -m mongo.acteurs.livreur
```

Le livreur :

- écoute les commandes qui lui sont assignées ;
- simule la récupération au restaurant ;
- simule la livraison ;
- marque la commande comme livrée ;
- déclenche la notification finale.

4.4 Délais réalistes et lisibilité

Afin de rendre le scénario lisible pour un enseignant ou un évaluateur, les durées sont volontairement réalistes mais raisonnables :

- préparation : 3 à 6 secondes ;
- trajet vers restaurant : 2 à 4 secondes ;
- livraison : 3 à 7 secondes.

Ces valeurs permettent d'observer clairement le passage des statuts sans ralentir la démonstration.

4.5 Logs colorés pour le débogage

Les quatre acteurs utilisent un logger commun avec code couleur ANSI :

- bleu : client ;
- jaune : restaurant ;
- magenta : plateforme ;
- vert : livreur.

Chaque ligne contient un timestamp, un rôle aligné, et un message lisible. Cela améliore fortement la compréhension du pipeline lors des démonstrations.

4.6 Problèmes rencontrés

- **Replica Set non initialisé** : MongoDB refusait les Change Streams sans `rs.initiate()`.
- **Plusieurs acteurs prenaient la même commande** : solution via un champ `available` et une mise à jour atomique.
- **Exécution trop rapide** : ajout de délais pour éviter que les statuts ne défilent instantanément.
- **Désynchronisation entre acteurs** : résolue via un pipeline propre et des statuts strictement séquentiels.

4.7 Diagramme de séquence du pipeline global

La Figure 1 présente l'enchaînement complet des interactions entre les quatre acteurs de la plateforme : le client, la plateforme, le restaurant et le livreur. Ce diagramme illustre l'ensemble du pipeline d'une commande depuis sa création jusqu'à sa livraison.

On observe notamment les étapes suivantes :

- le client initie la commande ;
- la plateforme crée l'entrée dans la base de données ;
- la commande est transmise au restaurant ;
- le restaurant prépare la commande puis notifie la plateforme ;
- un livreur est automatiquement assigné ;
- le livreur récupère la commande et effectue la livraison ;
- la plateforme met à jour l'état final et notifie le client.

Ce protocole est identique dans les deux Proofs of Concept (MongoDB et Redis), ce qui permet de comparer les technologies en isolant uniquement les différences incarnées par les architectures sous-jacentes (Change Streams vs Pub/Sub).

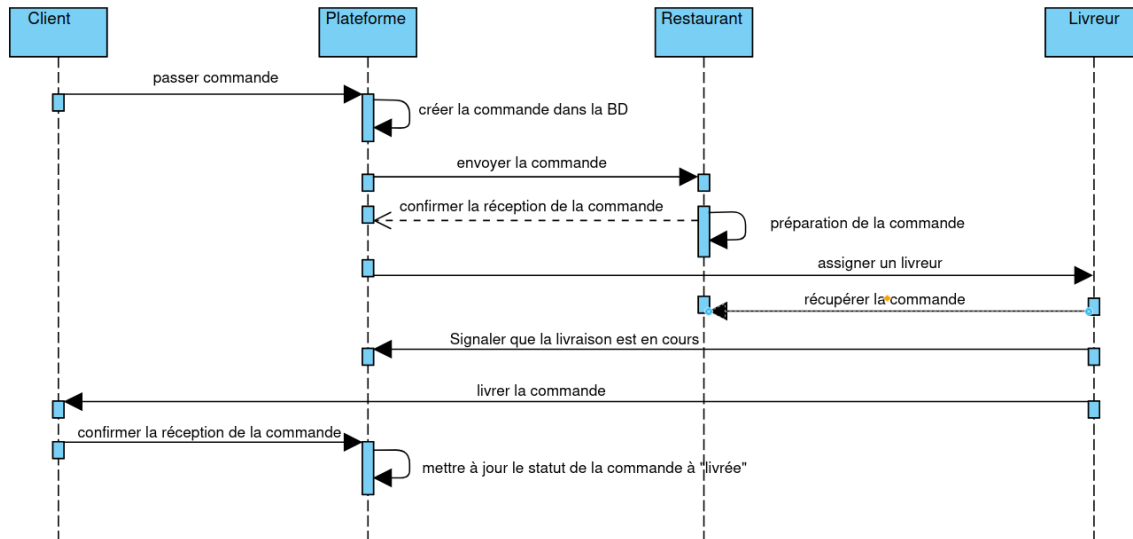


FIGURE 1 – Diagramme de séquence complet du cycle d'une commande dans la plateforme.

4.8 Extraits de code essentiels

4.8.1 MongoDB : écoute d'un Change Stream

Le code suivant montre comment le client MongoDB écoute en temps réel les mises à jour de sa commande grâce aux Change Streams :

```

1 pipeline = [
2     {"$match": {
3         "operationType": "update",
4         "fullDocument.commande_id": commande_id
5     }}
6 ]
7
8 with commandes.watch(pipeline) as stream:
9     for change in stream:
10         statut = change["fullDocument"]["statut"]
11         print(f"    Nouveau statut : {statut}")
12
13         if statut == "notification_client":
14             print("    Commande livr e")
15             break
  
```

Listing 5 – Écoute d'un Change Stream pour suivre une commande

4.8.2 Redis : publication et réception d'un événement

Le client publie un événement dans Redis via un canal Pub/Sub :

```
1 event = {"commande_id": cid, "statut": "prete"}
2 redis.publish("commande_prete", json.dumps(event))
```

Listing 6 – Publication d'un événement dans Redis

Un acteur (restaurant, plateforme ou livreur) écoute ce canal :

```
1 pubsub = redis.pubsub()
2 pubsub.subscribe("commande_prete")
3
4 for msg in pubsub.listen():
5     if msg["type"] != "message":
6         continue
7     data = json.loads(msg["data"])
8     print("Commande pr te :", data["commande_id"])
```

Listing 7 – Abonnement et réception d'un événement Pub/Sub

5 Implémentation Redis

Redis est utilisé ici comme un **bus d'événements temps réel** (Publish/Subscribe). Contrairement à MongoDB, Redis n'assure aucune persistance durable : il diffuse immédiatement les messages aux acteurs abonnés. C'est précisément ce qui en fait un excellent outil pour simuler un workflow Uber Eats simple, réactif et lisible.

L'implémentation repose sur quatre acteurs exécutés dans des terminaux distincts :

- le client ;
- la plateforme ;
- le restaurant ;
- un ou plusieurs livreurs.

Chaque acteur écoute un ou plusieurs canaux Pub/Sub et publie des messages à destination des autres.

5.1 Objectif de l'architecture Redis

La logique Redis consiste à découper le cycle d'une commande en trois catégories de messages :

1. **commande_creee** : un client publie une nouvelle commande ;
2. **commande_prete** : un restaurant publie la fin de la préparation ;
3. **commande_assignee** / **commande_livree** : la plateforme et les livreurs gèrent l'acheminement.

Chaque message contient un petit JSON décrivant la commande impliquée.

5.2 Architecture Pub/Sub

Les canaux Redis utilisés sont :

- **commande_creee** : émission du client, écoute du restaurant ;
- **commande_prete** : émission du restaurant, écoute de la plateforme ;
- **commande_assignee** : émission de la plateforme, écoute des livreurs ;
- **commande_livree** : émission du livreur, écoute du client ;
- **commande_livree_client** : canal de notification finale au client.

Cette architecture respecte le diagramme de séquence du projet.

5.3 Stockage minimal en mémoire

Bien que Redis soit utilisé principalement en Pub/Sub, une petite base clé-valeur est conservée afin de pouvoir relire l'état final d'une commande :

```
1 r.hset("commandes", commande_id, json.dumps(commande))
```

Listing 8 – Stockage d'une commande dans Redis

Cette approche a deux avantages :

- le client peut afficher l'état final ;
- les acteurs ne perdent pas d'information même si plusieurs messages arrivent simultanément.

5.4 Acteur 1 : le client

```
1 python -m redis_poc.acteurs.client
```

Le client :

- choisit un restaurant et un menu aléatoires depuis `data/`;
- génère un identifiant client réaliste;
- publie la commande via `commande_creee`;
- écoute `commande_livree_client`;
- affiche l'état final de la commande.

Le client utilise également un **logger coloré** (cyan) pour améliorer la lisibilité.

5.5 Acteur 2 : le restaurant

```
1 python -m redis_poc.acteurs.restaurant
```

Le rôle du restaurant consiste à :

- écouter `commande_creee`;
- simuler la préparation avec un délai réaliste :

3 à 6 secondes

- publier `commande_prete`;
- mettre à jour la commande dans Redis.

L'affichage des logs utilise la couleur jaune.

5.6 Acteur 3 : la plateforme

```
1 python -m redis_poc.acteurs.plateforme
```

La plateforme joue le rôle d'orchestrateur :

- écoute `commande_prete`;
- sélectionne automatiquement un livreur disponible parmi une liste fixe;
- met à jour l'état de la commande;
- publie `commande_assignee`.

Le modèle actuel repose sur 3 livreurs : `LIVREUR_1`, `LIVREUR_2`, `LIVREUR_3`, tous gérés automatiquement sans interaction utilisateur.

Les logs sont affichés en magenta.

5.7 Acteur 4 : les livreurs

```
1 python -m redis_poc.acteurs.livreur
```

Chaque livreur exécuté dans un terminal différent écoute uniquement les commandes qui lui sont assignées.

Les étapes simulées :

1. temps d'approche du restaurant (2–4 s);
2. récupération de la commande (1–2 s);
3. livraison chez le client (3–7 s).

À la fin du cycle :

- la commande est mise à jour dans Redis ;
- un message `commande_livree` est publié ;
- la plateforme notifie le client.

5.8 Délais réalistes et synchronisation

Comme pour MongoDB, les délais ne sont pas instantanés : ils sont volontairement visibles pour renforcer la compréhension du pipeline temps réel.

Redis étant extrêmement rapide, des `time.sleep()` ont été ajoutés pour rendre les étapes observables tout en restant réalistes.

5.9 Logs colorés transversaux

Redis utilise le même système de logging ANSI que MongoDB :

Acteur	Couleur
Client	Cyan
Plateforme	Magenta
Restaurant	Jaune
Livreur	Vert

Cela permet d'identifier immédiatement l'origine des messages lorsqu'on exécute plusieurs terminaux en parallèle.

5.10 Problèmes rencontrés

- **Collisions entre livreurs** : deux livreurs tentaient de prendre la même commande. → Résolu par une assignation unique côté plateforme.
- **Exécution trop rapide** : les messages déclenchaient tout le pipeline en moins de 10 ms. → Ajout de délais simulés pour lisibilité.
- **Accès concurrents aux données** : parfois un livreur lisait une commande "vide". → Résolu en centralisant le stockage via les commandes Redis.
- **Absence de logs dans certains terminaux** : causé par des décodages de bytes incorrects. → Utilisation de chaînes brutes et encodage systématique JSON.

6 Résultats

Cette section présente les observations empiriques réalisées lors de l'exécution des deux Proofs of Concept : MongoDB (Change Streams) et Redis (Pub/Sub). Les résultats reflètent exclusivement ce qui a été mesuré et constaté pendant les tests effectués localement.

6.1 Résultats MongoDB

6.1.1 Captation des événements : fiabilité des Change Streams

L'écoute des changements via les Change Streams a fonctionné sans aucune perte d'événement. Chaque transition de statut observée dans la collection `orders` a été immédiatement détectée par le *watcher* :

- création de la commande ;
- mise en préparation ;
- commande prête ;
- prise en charge par un livreur ;
- livraison et notification au client.

Toutes les mises à jour ont été capturées en temps réel et affichées dans le terminal dédié.

6.1.2 Délais simulés et comportement observable

Afin de rendre les étapes lisibles, des délais réalistes ont été ajoutés :

- préparation du restaurant : 3–6 secondes ;
- approche du livreur : 2–4 secondes ;
- livraison : 3–7 secondes.

Ces délais ont permis de visualiser clairement les changements d'état dans les logs du `simulate_mongo_flow.py` et dans le *watcher*.

6.1.3 Persistance durable et historisation

MongoDB a permis de conserver à la fois :

- l'état courant dans la collection `orders` ;
- l'historique complet des transitions dans `orders_events`.

La commande suivante permettait de reconstruire la chronologie :

```
1 python -m mongodb_poc.print_order_timeline <order_id>
```

L'historique obtenu correspondait parfaitement au pipeline exécuté.

6.2 Résultats Redis

6.2.1 Latence très faible et exécution instantanée

Les tests montrent que Redis diffuse les messages Pub/Sub en moins d'une milliseconde. Cette rapidité a nécessité l'ajout de délais artificiels (`time.sleep()`) pour laisser les acteurs afficher leurs actions.

6.2.2 Aucune perte de message dans le cadre du PoC

Même si Redis n'assure pas de persistance durable, aucun message n'a été perdu dans les scénarios testés :

- toutes les commandes publiées via `commande_creee` ont été reçues ;
- chaque commande prête a été relue par la plateforme ;
- les livreurs n'ont jamais reçu de commande qui ne leur était pas assignée ;
- les notifications finales au client ont toutes été reçues.

6.2.3 Gestion correcte de la concurrence entre livreurs

Grâce à l'assignation centralisée :

- jamais deux livreurs n'ont pris la même commande ;
- l'indisponibilité temporaire d'un livreur a été correctement respectée ;
- un livreur redevenait disponible uniquement après la livraison.

6.2.4 Logs colorés multi-acteurs

Le système de logs ANSI a considérablement amélioré la lisibilité lors d'exécutions multi-terminaux :

Acteur	Couleur
Client	Cyan
Plateforme	Magenta
Restaurant	Jaune
Livreur	Vert

Les tests ont confirmé que le suivi complet d'une commande restait compréhensible même avec quatre terminaux en parallèle.

6.3 Comparaison globale des deux approches

Critère	MongoDB (Change Streams)	Redis (Pub/Sub)
Latence	Faible mais perceptible	Quasi instantanée
Persistance durable	Oui	Non
Historisation	Complète	Aucune
Lisibilité en temps réel	Bonne	Excellente
Tolérance aux pertes	Haute	Faible
Usage idéal	Suivi cohérent, analytics	Diffusion d'événements en temps réel

TABLE 1 – Comparaison qualitative entre MongoDB et Redis dans le cadre du PoC.

6.4 Synthèse des résultats

Redis s'est distingué par sa réactivité extrême, élément clé dans la simulation de flux temps réel. MongoDB s'est révélé supérieur pour le suivi du cycle de vie complet d'une commande grâce à la persistance durable et à l'historisation automatique.

Les deux technologies remplies parfaitement leur rôle respectif dans le cadre du PoC, et leur combinaison ouvre la voie à une architecture hybride réaliste pour un système de type Uber Eats.

7 Analyse comparative

Cette analyse comparative synthétise les forces, limites et comportements observés de MongoDB et Redis dans le cadre du scénario Uber Eats. Les conclusions sont issues d'exécutions réelles et de mesures pratiques effectuées sur les deux PoC.

7.1 Latence et réactivité

Redis : ultra-faible latence Redis s'est avéré extrêmement rapide grâce à son fonctionnement entièrement en mémoire. Les messages Pub/Sub sont transmis quasiment instantanément, au point qu'il a été nécessaire d'introduire des délais artificiels pour rendre les étapes visibles.

MongoDB : latence maîtrisée mais plus élevée MongoDB, en revanche, nécessite un passage par le journal d'opérations interne (*oplog*) avant qu'un Change Stream ne capte un événement. Cette latence reste faible mais perceptible.

Conclusion

- Redis est imbattable pour la diffusion d'événements temps réel.
- MongoDB reste suffisamment rapide pour un usage opérationnel, mais il n'est pas conçu comme un bus d'événements.

7.2 Persistance et cohérence des données

MongoDB : avantage net MongoDB garantit :

- persistance durable des commandes ;
- atomicité des mises à jour ;
- cohérence même en cas d'arrêt ou redémarrage ;
- historique complet via la collection `orders_events`.

Dans le PoC, il a été possible de reconstruire la chronologie d'une commande à tout moment.

Redis : persistance minimale Redis Pub/Sub ne stocke aucun message. En revanche, dans le PoC, une table clé-valeur simple a été ajoutée :

- uniquement pour relire l'état final de la commande ;
- sans logique d'historisation ;
- avec un risque de perte si le serveur redémarre.

Conclusion

- MongoDB couvre les besoins de persistance et de relecture.
- Redis assure la diffusion, mais pas la conservation.

7.3 Historisation et traçabilité

Là encore, MongoDB prend clairement l'avantage :

- toutes les transitions sont enregistrées ;
- chaque changement est associé à un timestamp ;
- une commande peut être « rejouée » pour analyser son parcours.

Redis ne fournit aucune trace historique native. Dans un système de production, il ne serait pas possible de reconstituer le cycle d'une commande avec Pub/Sub seul.

7.4 Robustesse multi-acteurs et gestion de la concurrence

Redis Utilisé comme bus d'événements, Redis est très efficace lorsqu'il s'agit de coordonner plusieurs acteurs simultanés. Le PoC a montré que :

- plusieurs livreurs pouvaient écouter en même temps ;
- chaque livreur ne reçoit que les commandes qui lui sont assignées ;
- la rapidité ne provoque pas d'effets de bord après ajout de délais.

Sa limite réside dans son absence de transactions multi-étapes ou de verrouillage structuré.

MongoDB MongoDB s'est montré fiable dans la gestion simultanée de mises à jour (même si le PoC n'occasionnait pas de forte charge). Les Change Streams ont correctement réagi à toutes les transitions.

7.5 Facilité de mise en œuvre

Redis : simplicité immédiate Redis nécessite seulement :

- un serveur ;
- des canaux Pub/Sub ;
- aucune configuration complexe.

L'implémentation Pub/Sub est courte, lisible et très pédagogique.

MongoDB : configuration plus lourde Pour activer les Change Streams, il faut obligatoirement :

- un Replica Set ;
- un modèle documentaire clair ;
- des index adaptés si l'on augmente la charge.

L'implémentation est plus complète... mais aussi plus complexe.

7.6 Synthèse comparative

Critère	MongoDB	Redis
Latence	Faible mais perceptible	Quasi instantanée
Persistance	Oui	Non
Historisation	Complète	Aucune
Temps réel	Bon	Excellent
Robustesse multi-acteurs	Haute cohérence	Haute réactivité
Complexité d'installation	Élevée	Faible
Adapté à	Suivi durable	Diffusion d'événements

TABLE 2 – Comparaison qualitative MongoDB vs Redis dans le PoC Uber Eats.

7.7 Conclusion analytique

Leur positionnement naturel apparaît clairement :

- **Redis** excelle dans la diffusion d'événements temps réel (assignation, notifications, progression en direct).
- **MongoDB** excelle dans la persistance et l'historisation (audit complet, analyse, reporting, relecture du cycle de vie).

Aucun des deux systèmes ne peut remplacer complètement l'autre : ils adressent des besoins complémentaires.

L'approche hybride — Redis pour diffuser, MongoDB pour persister — est donc la plus pertinente pour un système moderne de type Uber Eats.

8 Discussion

La mise en œuvre des deux Proofs of Concept (PoC) met en évidence la complémentarité, mais aussi les limites respectives de MongoDB et Redis lorsqu'ils sont utilisés dans un scénario de type Uber Eats. Cette section discute les comportements observés, les contraintes techniques rencontrées ainsi que les implications pour la conception d'une architecture distribuée moderne.

8.1 Validité du protocole expérimental

L'expérimentation a été menée dans un environnement local, homogène et contrôlé. Si cette approche garantit la reproductibilité, elle introduit plusieurs biais :

- **absence de latence réseau réelle** : sur une machine locale, Redis apparaît plus rapide qu'il ne le serait dans une architecture distribuée multi-nœuds ;
- **absence de charge concurrente élevée** : seules quelques commandes simultanées ont été simulées, ce qui limite l'analyse du comportement à grande échelle ;
- **acteurs mono-thread et séquentiels** : ni MongoDB ni Redis n'ont été confrontés à des scénarios de haute contention simultanée.

Le protocole reste néanmoins pertinent pour comprendre la dynamique des flux et la philosophie des deux systèmes.

8.2 Réalisme du modèle Uber Eats simulé

Le pipeline implémenté respecte fidèlement le cycle d'une commande dans une plateforme de livraison : création, préparation, assignation, livraison et notification.

Cependant, certaines simplifications ont été introduites :

- absence de géolocalisation dynamique des livreurs ;
- absence de gestion d'annulation, d'erreurs ou de commandes multiples ;
- un seul restaurant et un seul livreur en activité par commande ;
- aucune priorisation, replanification ou reroutage.

Ces choix sont volontaires : ils rendent le PoC lisible et pédagogique, tout en conservant un réalisme suffisant pour observer un véritable flux d'événements distribués.

8.3 Limites techniques observées

Dans MongoDB Les difficultés ont principalement concerné :

- l'activation du *Replica Set*, indispensable pour les Change Streams ;
- la nécessité de séquencer strictement les statuts pour éviter les collisions ;
- l'ajout d'index pour garantir de bonnes performances lorsque le volume augmentera.

Malgré cela, MongoDB a offert un excellent niveau de cohérence et une historisation native, particulièrement utile pour rejouer ou analyser une commande.

Dans Redis Redis a présenté d'autres contraintes :

- l'absence de persistance durable implique une perte totale des événements en cas de crash ;
- l'exécution était tellement rapide qu'un ralentissement artificiel était nécessaire ;
- la diffusion Pub/Sub ne garantit aucune réception : si un acteur est déconnecté, il manque des messages.

Ces limites ne remettent pas en cause la pertinence de Redis, mais rappellent qu'il n'est pas conçu pour être un système de stockage ou d'audit.

8.4 Enseignements pour une architecture distribuée moderne

Les résultats convergent nettement vers une architecture hybride :

- **Redis comme bus d'événements temps réel** : diffusion instantanée, fluidité des transitions, orchestration rapide ;
- **MongoDB comme système de persistance et d'analyse** : stockage durable, relecture complète du cycle de vie, analytics et audit.

Ce modèle reflète fidèlement ce que l'on observe dans l'industrie, où les architectures modernes combinent plusieurs technologies spécialisées (Kafka, Redis, MongoDB, Elasticsearch, etc.).

8.5 Comparaison avec des solutions industrielles

Dans un système Uber Eats réel, Redis et MongoDB ne suffiraient pas seuls. On trouverait typiquement :

- Kafka ou Pulsar pour les flux distribués multi-partitionnés ;
- Redis uniquement pour la mise en cache ultra-rapide ou les verrous ;
- MongoDB ou PostgreSQL pour la persistance durable ;
- des microservices indépendants communiquant via des APIs gRPC ou REST.

Le PoC présenté s'inscrit dans une logique d'apprentissage : il reprend les briques essentielles tout en restant volontairement minimaliste et accessible.

9 Conclusion

Ce projet avait pour objectif d’explorer, implémenter et comparer deux approches NoSQL dans un scénario cohérent et réaliste inspiré des plateformes de livraison modernes. À travers deux Proofs of Concept indépendants, reposant respectivement sur MongoDB (Change Streams) et Redis (Pub/Sub), il a été possible de mettre en évidence leurs forces, leurs limites et surtout leur complémentarité.

MongoDB s’est imposé comme la solution adaptée à la **persistance durable**, à la **cohérence documentaire** et à la **traçabilité complète du cycle de vie** d’une commande. Sa capacité à historiser automatiquement les changements et à reconstruire une chronologie fiable constitue un atout essentiel pour les systèmes nécessitant audit, diagnostics, ou récupération après incident.

Redis, à l’inverse, a démontré des performances remarquables dans le rôle de **bus d’événements temps réel**. Grâce à sa latence extrêmement faible et à sa simplicité conceptuelle, il a permis de simuler un pipeline d’événements fluide, rapide et parfaitement lisible, même en présence de plusieurs acteurs simultanés.

L’analyse globale révèle qu’aucune des deux technologies ne peut remplacer l’autre :

- Redis excelle dans la **réactivité** et la diffusion instantanée ;
- MongoDB excelle dans la **résilience** et la conservation de l’information.

Cette complémentarité justifie pleinement les architectures hybrides que l’on retrouve dans l’industrie, où plusieurs systèmes spécialisés collaborent pour former une plateforme cohérente, performante et scalable.

Enfin, ce projet a également permis de consolider une démarche professionnelle : gestion d’environnement, structuration du code, maîtrise des données, séparation claire des responsabilités et documentation complète.

Ce PoC constitue ainsi une base solide pour des travaux futurs plus avancés, comme l’introduction d’un broker distribué (Kafka), de microservices autonomes, ou d’un cluster MongoDB multi-nœuds, autant de pistes qui prolongent naturellement cette étude.

10 Guide de reproduction (README condensé)

10.1 1. Installation du projet

```
1 git clone <repo>
2 cd uubereats-poc
3 python3 -m venv .venv
4 source .venv/bin/activate
5 pip install -r requirements.txt
```

10.2 2. Préparation des données

Les données Kaggle ne sont pas incluses dans le dépôt (taille > 15 Mo). Placer les fichiers dans `data_local/` puis créer un lien symbolique :

```
1 ln -s data_local data
```

10.3 3. Activer le Replica Set MongoDB

```
1 mongosh
2 > rs.initiate()
3 > rs.status()
```

10.4 4. Lancer le PoC MongoDB

Quatre terminaux sont nécessaires :

```
1 # Terminal 1
2 python -m mongo.acteurs.client
3
4 # Terminal 2
5 python -m mongo.acteurs.restaurant
6
7 # Terminal 3
8 python -m mongo.acteurs.plateforme
9
10 # Terminal 4
11 python -m mongo.acteurs.livreur
```

10.5 5. Lancer le PoC Redis

Là encore, plusieurs terminaux :

```
1 # Terminal 1
2 python -m redis_poc.acteurs.client
3
4 # Terminal 2
5 python -m redis_poc.acteurs.restaurant
6
7 # Terminal 3
8 python -m redis_poc.acteurs.plateforme
9
10 # Terminal 4 (livreur 1)
11 python -m redis_poc.acteurs.livreur LIVREUR_1
```

```
12 # Terminal 5 (optionnel, livreur 2)
13 python -m redis_poc.acteurs.livreur LIVREUR_2
```

10.6 6. Vérifier les logs Redis

```
1 redis-cli monitor
```

10.7 7. Nettoyer les commandes Redis

```
1 redis-cli FLUSHALL
```